Name - Chandra Prakash

Roll no - 23010102 7

GitHub Repository - https://github.com/cp18112004/cloud-computing-assignment

Project: Knowledge Deployment of a Lightweight Command-Parsing Language Model on a Local Cloud

1. What I Learnt

→ Knowledge Distillation (KD) in Practice:

I learnt how sequence-level KD function by training a smaller student model directly on outputs of larger teacher model rather than requiring logits. The teacher (phi3.5, 3.8B parameters) generated structured four-key JSON targets for 3000 commands. Using the first 1500 cleaned records, I fine-tuned Eleuther AI/Pythia-70m on CPU. The student achieved 100% JSON validity and 50% full frame accuracy (compared to 53% for the teacher & 0% for the student base model). Logit-based distillation was not possible because the model use different vocabularies/tokenizers (phi-base model). Logit-based distillation was not possible because the models use different vocabularies/tokenizers (Phi-3.5 has 32,064 tokens vs Pythia's 50,304 tokens) and Ollama only return text completi →

→ Reproducible Data Processing & Sharding

I implemented deterministic data partitioning using modular arithmetic (line_number % SHARD_COUNT == SHARD_INDEX),

allowing parallel workers to process independent slices. without coordination or locking. I learnt the importance of deterministic filtering (clean.py) and checksum verification (train.sha256) to ensure reproducible training inputs.

→ Containerization & Kubernetes Architecture

I learnt how to design lightweight container images by excluding model weights (keeping the service image at ~0.95-GB) and dynamically fetching weights from S3/SeaweedFS at container startup. I understand the critical role of Kubernetes probes: /ready2 returning HTTP 503 during model weight download prevents traffic from hitting uninitialized pods, while /healthz provides liveness checks.

2. Challanges and How I solved them

→ Show Teacher on CPU (~8-10 per command):

Labelling 3000 commands on CPU would take ~3.5 hours. I solved this by sharding the dataset across parallel workers, which ~~pos~~ processed independent slices concurrently and drastically reduced runtime.

→ Teacher Output Inconsistencies:

The teacher model initially appended conversational text (eg. (Notes: ...)) after the JSON. Adding a newline stop sequence in Ollama forced it to emit only a clean, single-line JSON, reading 100% validity.

→ Strict 1 CPU & 1 GiB RAM Limit:

Running PyTorch within 1 GiB risked OOM kills. I used the light weight CPU-only PyTorch wheel and serialized inference requests. Peak memory stayed at 657 MiB with zero OOM errors across 50 requests

3. Key Obeservations & Finding

→ Model Performance

The 70M student achieved 50% full-frame accuracy and 100% JSON validity, reaching 86% of the 3.8B teacher's performance (58%). An extra epoch on model v2 caused slight overfitting (99% validity). So v1 was kept as the final model.

→ Cost & SLO Analysis

Running two replicas costs ₹7.90/hours (₹0.469 per 1000 requests). The 99% success SLO was fully satisfied with 0 failed requests out of 5 57 (0% error budget used).

4. Limitations & Future improvements

→ Single-Node Cluster: k3d runs on single node, which does not provide true fault tolerance. Production needs a multi-node cluster with Pod Anti-Affinity.

→ Cold Starts: Pods take ~14s to download 282 MB of weights from S3 at start up. A node-local model cache would significantly cut this startup time.

→ Constrained Decoding: The student occasionally copies teachers mistakes. Grammar / schema-constrained decoding would guarantee strict JSON syntax.